SESSION 4: UNSUPERVISED LEARNING

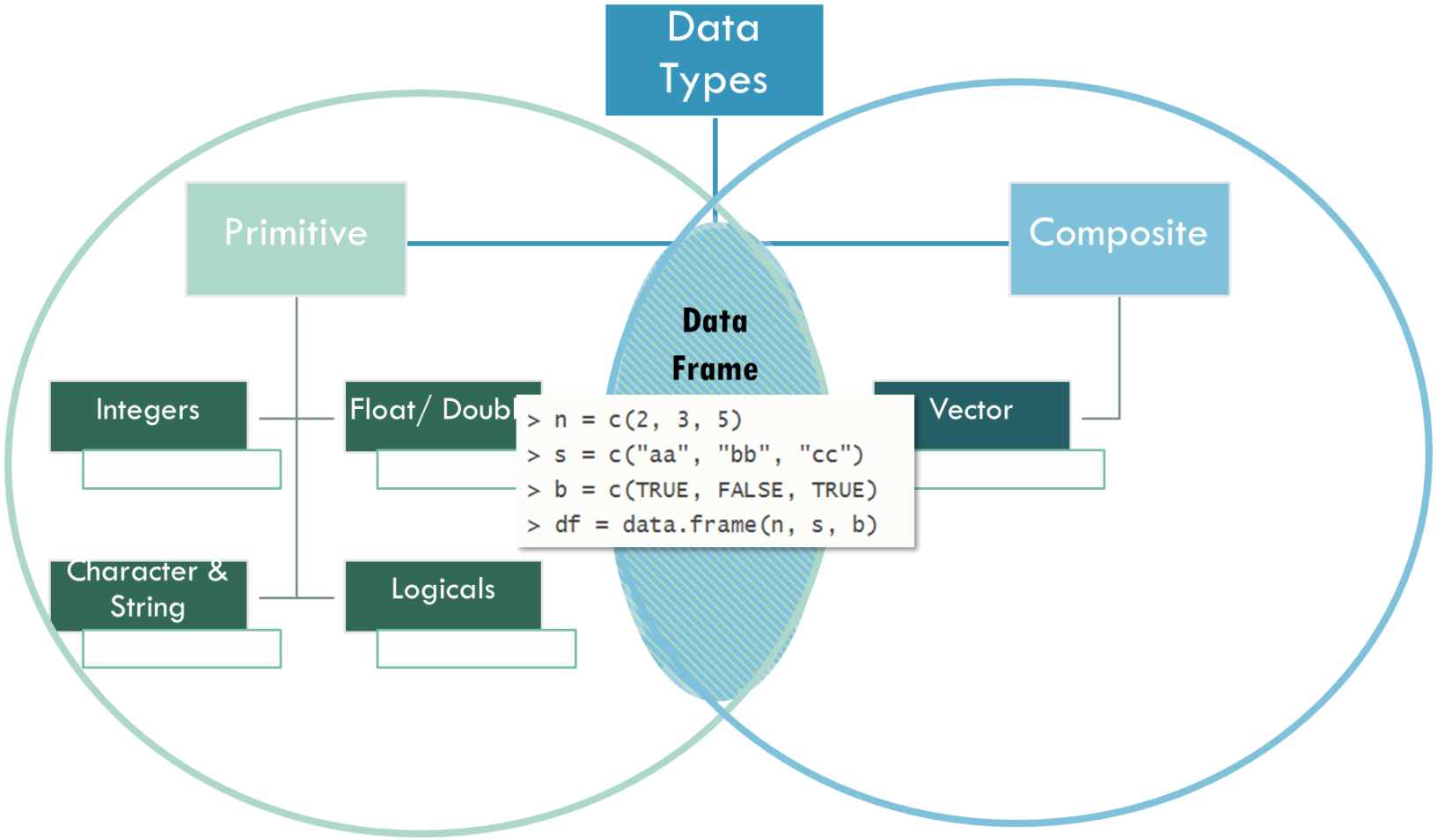
DR. SOFIA GIL-CLAVEL

- The basics of Statistical Learning
- Unsupervised Methods
- PCA
- K-means

WHAT HAVE YOU LEARNED SO FAR?

- R and RStudio
- Data types
- Data frames

Introduction to R and Data frames



Introduction to R and Data frames

Using packages: Tidyverse

- Packages
- Tidy logic
- Dplyr

1.

```
> iris
# A tibble: 150 × 5
   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
          <dbl>       <dbl>        <dbl>       <dbl> <fct>
 1          5.1         3.5          1.4         0.2 setosa
 2          4.9         3            1.4         0.2 setosa
 3          4.7         3.2          1.3         0.2 setosa
 4          4.6         3.1          1.5         0.2 setosa
 5          5           3.6          1.4         0.2 setosa
 6          5.4         3.9          1.7         0.4 setosa
 7          4.6         3.4          1.4         0.3 setosa
 8          5           3.4          1.5         0.2 setosa
 9          4.4         2.9          1.4         0.2 setosa
10          4.9         3.1          1.5         0.1 setosa
# ℹ 140 more rows
# ℹ Use `print(n = ...)` to see more rows
```

2.

Basic piping

- `x %>% f` is equivalent to `f(x)`
- `x %>% f(y)` is equivalent to `f(x, y)`
- `x %>% f %>% g %>% h` is equivalent to `h(g(f(x)))`

3.

dplyr is a grammar of data manipulation, providing a consistent set of verbs that help you solve the most common data manipulation challenges:

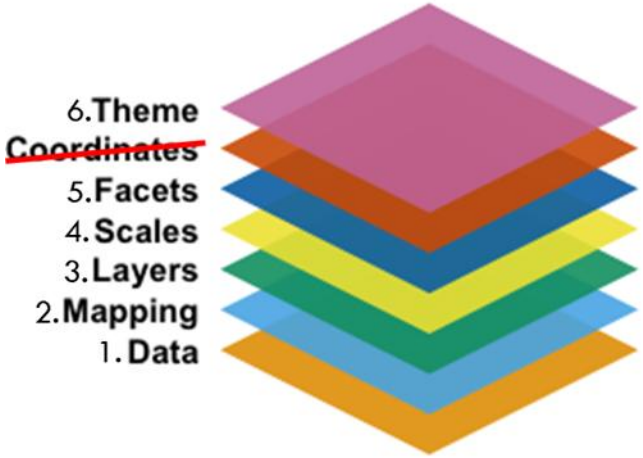
- `mutate()` adds new variables that are functions of existing variables
- `select()` picks variables based on their names.
- `filter()` picks cases based on their values.
- `summarise()` reduces multiple values down to a single summary.
- `arrange()` changes the ordering of the rows.

Introduction to R and Data frames

Using packages: Tidyverse

- Grammar of graphs
- Ggplot2
- Graphs components

Data Visualizations with ggplot2




```
ggplot(mpg, aes(x = displ, y = hwy,
                colour = class
                shape = drv
                size = cyl
                )) +
geom_point()
```

1.Data

2. Mapping: Variables in the Data

4. Scales

Add and transform the graph

3. Scale: Geometric Object

Introduction to R and Data frames

Using packages: Tidyverse

Data Visualizations with ggplot2

Unsupervised Learning

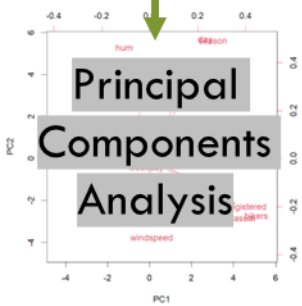
- 1.1 Statistical learning
- 1.2 Supervised vs. Unsupervised
- 1.3 PCA
- 1.4 K-means

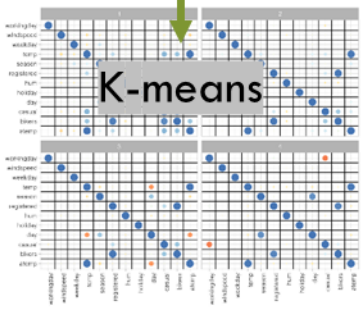
$$\cancel{Y_i} \sim X1_i + X2_i + X3_i$$

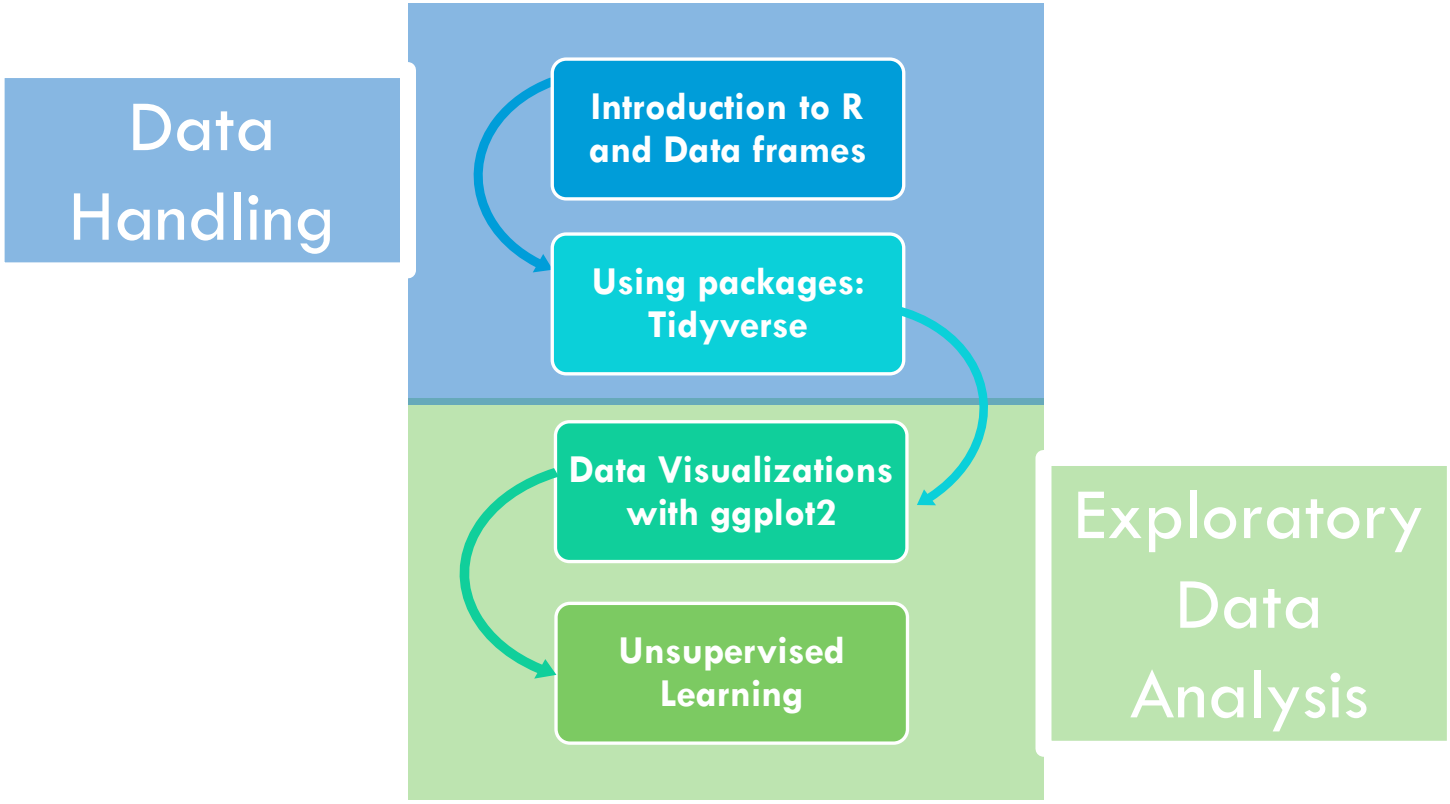
Numeric Matrix

Dimensionality reduction

Clustering






Data Handling
Introduction to R and Data frames
Using packages: Tidyverse
Data Visualizations with ggplot2
Unsupervised Learning
Exploratory Data Analysis

BREAK

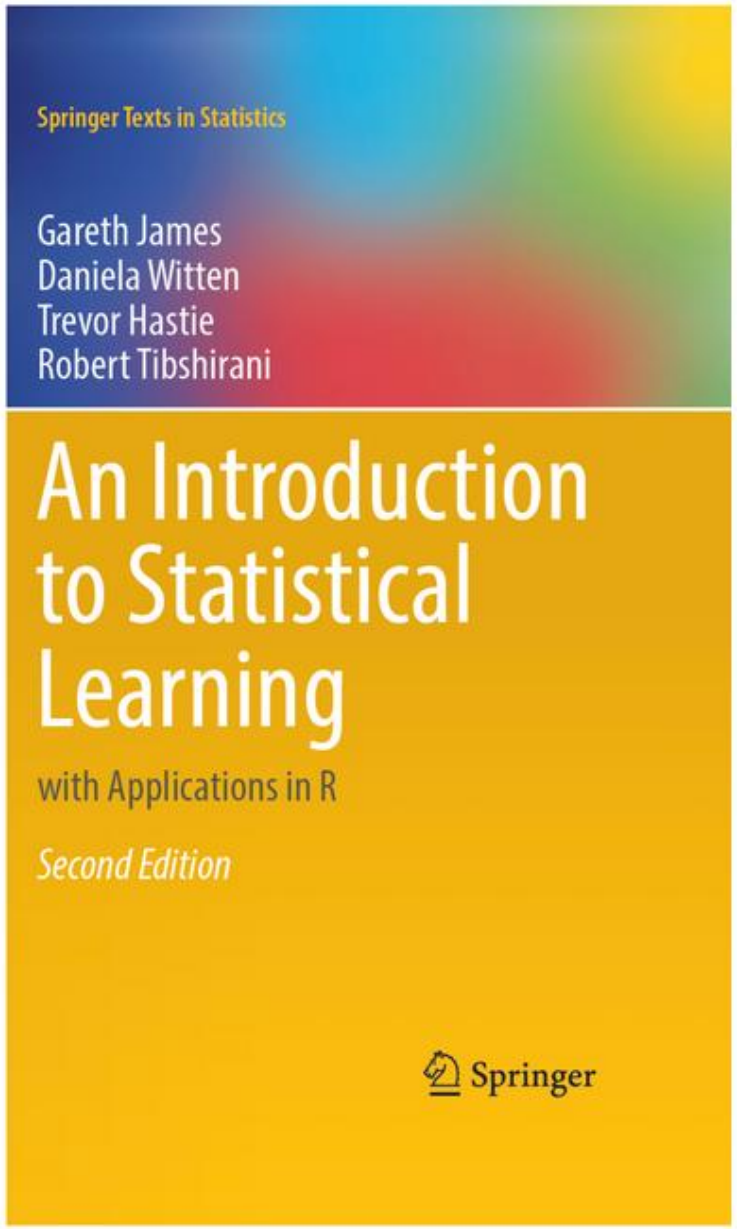
See you in 10 mins.

1. THE BASICS OF STATISTICAL LEARNING

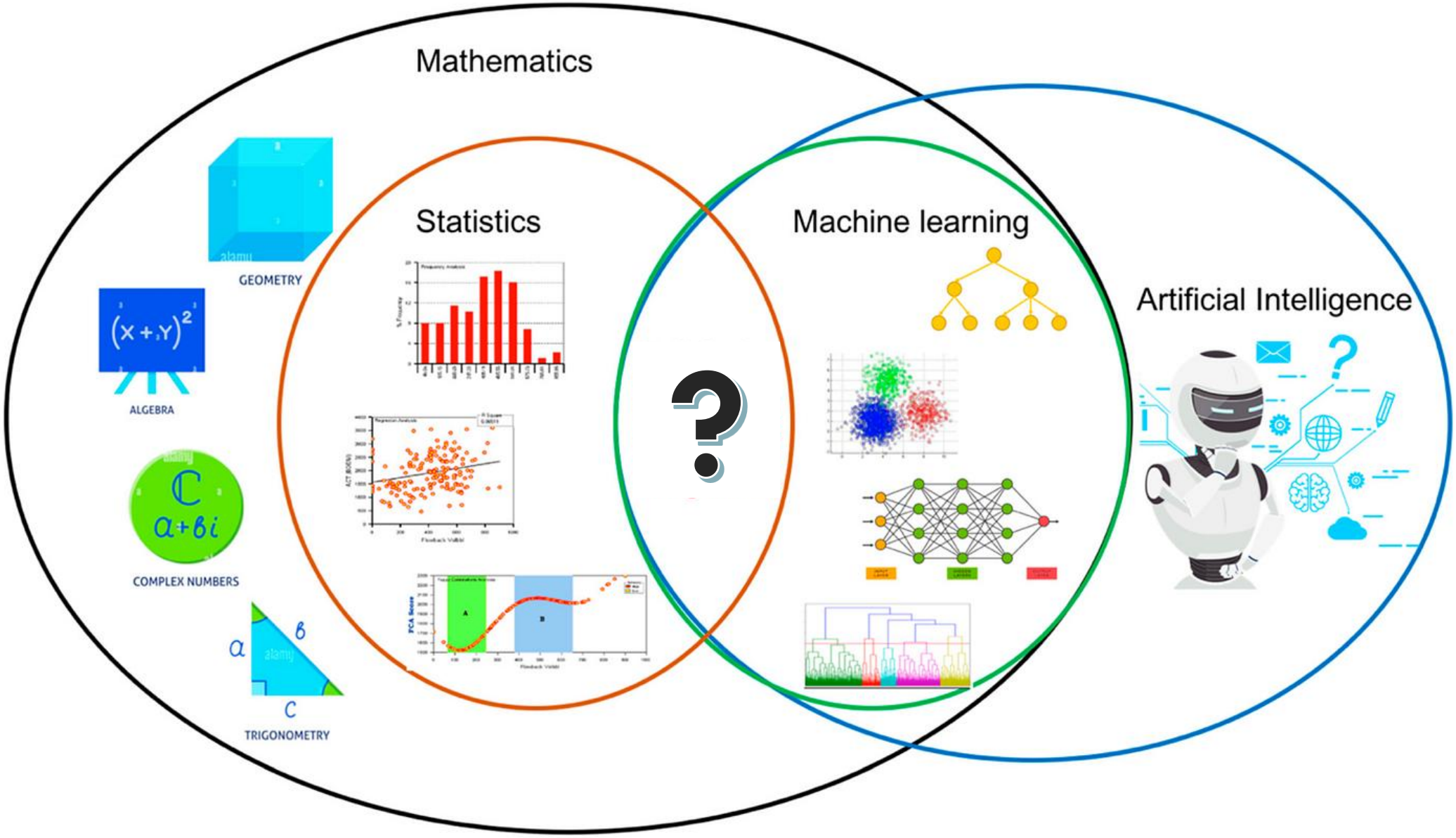
1.1 Statistical learning
1.2 Supervised vs. Unsupervised
1.3 Overview of unsupervised methods

WE WILL BE FOLLOWING:

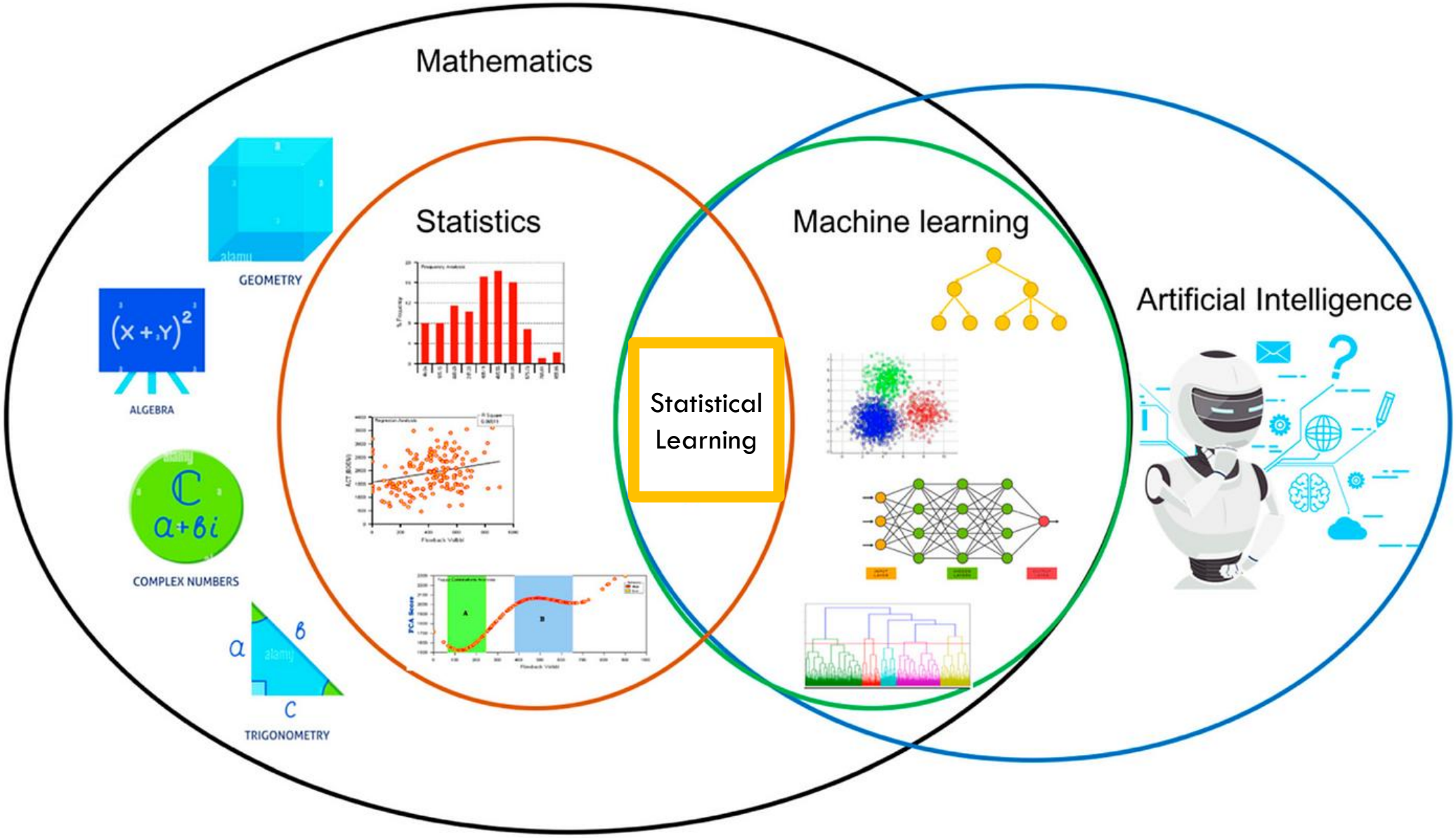
You can find the book for free here:
https://www.statlearning.com/



1.1 STATISTICAL LEARNING



Original source: https://medium.com/stats-learning/differences-and-synergies-between-statistics-and-machine-learning-90cea85d4cf5



Original source: https://medium.com/stats-learning/differences-and-synergies-between-statistics-and-machine-learning-90cea85d4cf5

WHY STATISTICAL LEARNING?

So far in these workshops we have learned to handle data frames in R.

data frame name

columns

```
> Bikeshare
# A tibble: 8,645 × 12
   bikers season   day holiday weekday workingday  temp atemp   hum
    <dbl>  <dbl> <dbl>   <dbl>   <dbl>      <dbl> <dbl> <dbl> <dbl>
 1     16      1     1       0       6          0  0.24 0.288  0.81
 2     40      1     1       0       6          0  0.22 0.273  0.8
 3     32      1     1       0       6          0  0.22 0.273  0.8
 4     13      1     1       0       6          0  0.24 0.288  0.75
 5      1      1     1       0       6          0  0.24 0.288  0.75
 6      1      1     1       0       6          0  0.24 0.258  0.75
 7      2      1     1       0       6          0  0.22 0.273  0.8
 8      3      1     1       0       6          0  0.2  0.258  0.86
 9      8      1     1       0       6          0  0.24 0.288  0.75
10     14      1     1       0       6          0  0.32 0.348  0.76
# ℹ 8,635 more rows
# ℹ 3 more variables: windspeed <dbl>, casual <dbl>,
#   registered <dbl>
# ℹ Use `print(n = ...)` to see more rows
```

rows

variable

value/element

WHY STATISTICAL LEARNING?

So far in these workshops we have learned to handle data frames in R. This is very convenient, as we can analyze these data frames using statistical symbolic language!

- **Dependent** (Y): Also known as response variable.
- **Independent** (X): Also known as input, predictor, feature, or just variable.

```
> Bikeshare
# A tibble: 8,645 × 12
   bikers season   day holiday weekday workingday  temp atemp   hum
   <dbl>  <dbl> <dbl>   <dbl>   <dbl>      <dbl> <dbl> <dbl> <dbl>
 1     16      1     1       0       6          0  0.24 0.288  0.81
 2     40      1     1       0       6          0  0.22 0.273  0.8
 3     32      1     1       0       6          0  0.22 0.273  0.8
 4     13      1     1       0       6          0  0.24 0.288  0.75
 5      1      1     1       0       6          0  0.24 0.288  0.75
 6      1      1     1       0       6          0  0.24 0.258  0.75
 7      2      1     1       0       6          0  0.22 0.273  0.8
 8      3      1     1       0       6          0  0.2  0.258  0.86
 9      8      1     1       0       6          0  0.24 0.288  0.75
10     14      1     1       0       6          0  0.32 0.348  0.76
# ℹ 8,635 more rows
# ℹ 3 more variables: windspeed <dbl>, casual <dbl>,
#   registered <dbl>
# ℹ Use `print(n = ...)` to see more rows
```

dependent variable (Y)

independent variables (X)

WHY STATISTICAL LEARNING?

So far in these workshops we have learned to handle data frames in R. This is very convenient, as we can analyze these data frames using statistical symbolic language!

- **Dependent** (Y): Also known as response variable.
- **Independent** (X): Also known as input, predictor, feature, or just variable.

Symbolic Language

$$Y \sim X1 + X2 + X3$$

R was built to understand symbolic language!

1.2 UNSUPERVISED VS. SUPERVISED

SUPERVISED STATISTICAL LEARNING

In supervised learning, for each observation of the predictor measurement(s) X_i there is an associated response measurement Y_i.

$$Y_i \sim X1_i + X2_i + X3_i$$

i refers to the data frame row

We wish to fit a model that relates the response to the predictors, with the aim of:

- Prediction: accurately predicting the response for future observations.
- Inference: better understanding the relationship between the response and the predictors.

Many classical statistical learning methods such as linear regression and logistic regression, as well as more modern approaches such as GAM, boosting, and support vector machines, operate in the supervised learning domain.

UNSUPERVISED STATISTICAL LEARNING

Unsupervised learning describes the somewhat more challenging situation in which for every observation i = 1,…, n, we observe a vector of measurements X_i but no associated response Y_i.

$$\cancel{Y_i} \sim X1_i + X2_i + X3_i$$

We are not interested in prediction, because we do not have an associated response variable Y. The goal is to discover interesting things about the measurements on X1, X2,…, Xp. Is there an informative way to visualize the data? Can we discover subgroups among the variables or among the observations?

Unsupervised learning is often performed as part of an exploratory data analysis.

1.3 OVERVIEW OF UNSUPERVISED METHODS

WE WILL FOCUS ON TWO KINDS OF UNSUPERVISED LEARNING:

- **Dimensionality reduction:** a tool to reduce the dimensionality of your data. It is used for data visualization or data pre-processing (e.g., data imputation) before supervised techniques are applied.
  - Principal Components Analysis (PCA): looks to find a low-dimensional representation of the observations that explain a good fraction of the variance.

- **Clustering:** Clustering refers to a very broad set of techniques for finding subgroups, or clusters, in a data set. When we cluster the observations of a data set, we seek to partition them into distinct groups so that the observations within each group are quite similar to each other, while observations in different groups are quite different from each other.
  - K-means: we seek to partition the observations into a pre-specified number of clusters.

OUR MAIN GOAL TODAY

We aim to identify cyclists with similar cycling behavior. Perhaps, we can start offering services based on their commonalities.

For this, we will use the following R packages, **please install them and load them:**

- tidyverse – You should already have it.
- ISLR2
- corrplot
- scatterplot3d

THE "BIKESHARE" DATA

Save a copy of the data as a tibble.

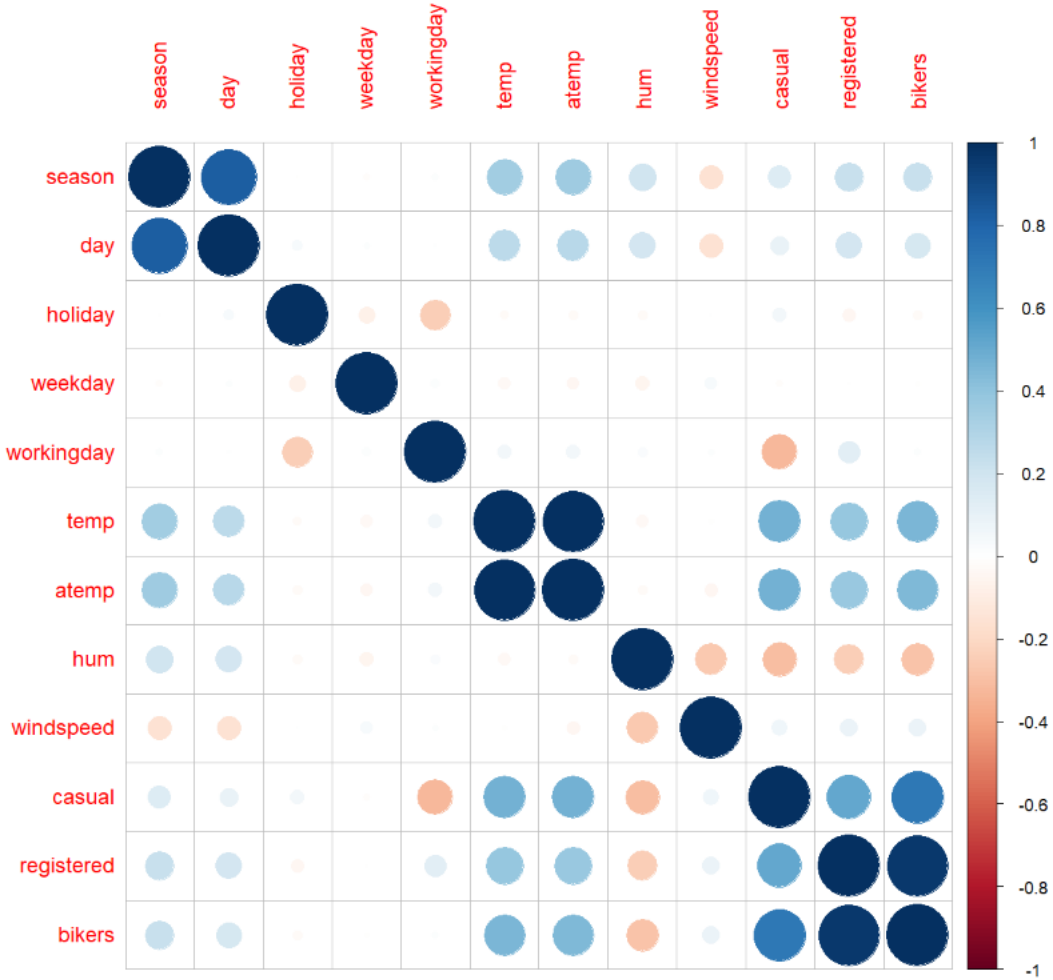
- **season:** Season of the year, coded as Winter=1, Spring=2, Summer=3, Fall=4.
- **mnth:** Month of the year, coded as a factor.
- **day:** Day of the year, from 1 to 365
- **hr:** Hour of the day, coded as a factor from 0 to 23.
- **holiday:** Is it a holiday? Yes=1, No=0.
- **weekday:** Day of the week, coded from 0 to 6, where Sunday=0, Monday=1, Tuesday=2, etc.
- **workingday:** Is it a work day? Yes=1, No=0.
- **weathersit:** Weather, coded as a factor.
- **temp:** Normalized temperature in Celsius. The values are derived via (t-t_min)/(t_max-t_min), t_min=-8, t_max=+39.
- **atemp:** Normalized feeling temperature in Celsius. The values are derived via (t-t_min)/(t_max-t_min), t_min=-16, t_max=+50.
- **hum:** Normalized humidity. The values are divided to 100 (max).
- **windspeed:** Normalized wind speed. The values are divided by 67 (max).
- **casual:** Number of casual bikers.
- **registered:** Number of registered bikers.
- **bikers:** Total number of bikers.

THE "BIKESHARE" DATA

EXERCISE 1.1:

1. Let's keep only the variables that are numerical.
2. Now, let's visualize their correlations.

   Hint: Use the function corrplot() with the argument/parameter method="circle".
3. What can you tell about the data?



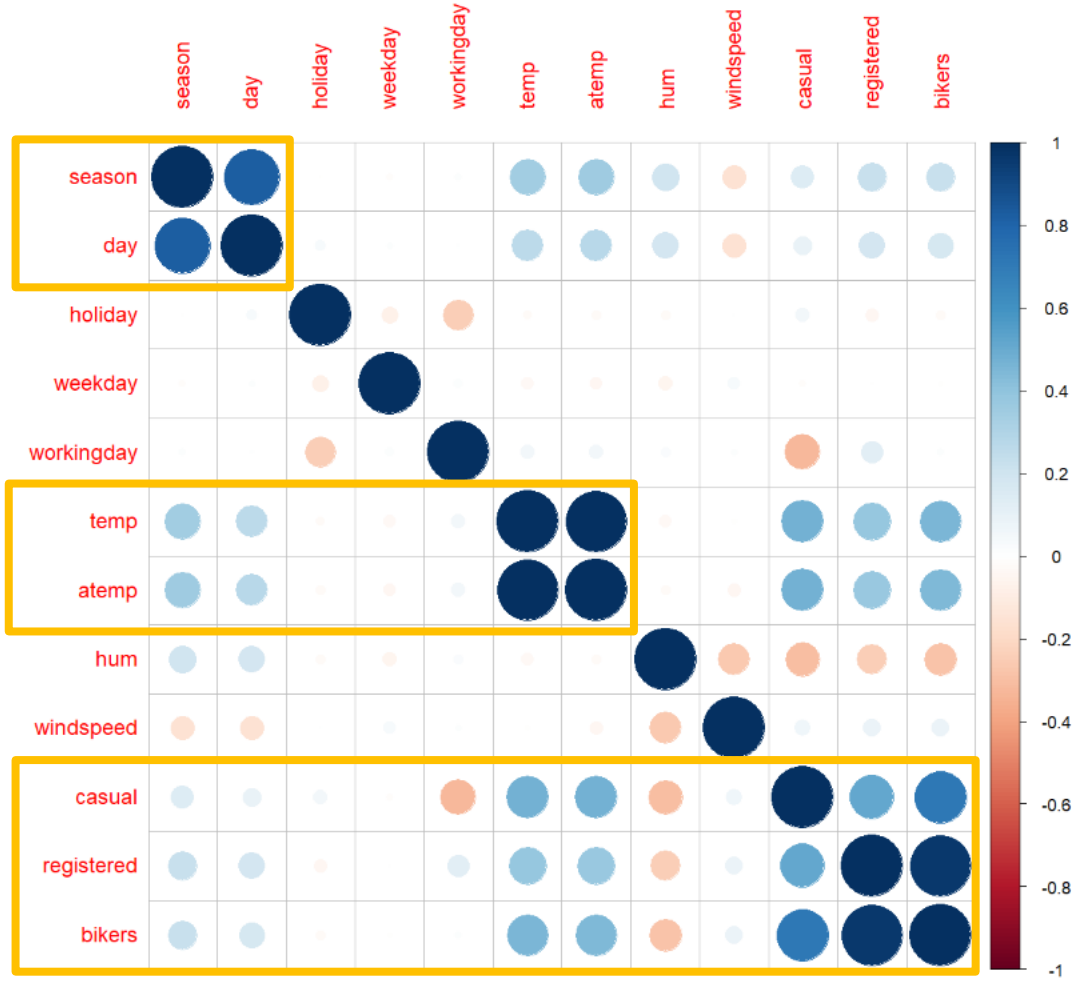
3. PRINCIPAL COMPONENTS ANALYSIS

1. Introduction
2. Visualizing dimensionality reduction

WHAT IS PRINCIPAL COMPONENTS?

When faced with a large set of **correlated variables**, **principal components allow us to summarize this set with a smaller number of** representative **variables** that collectively explain most of the variability in the original set.

THE “BIKESHARE” DATA

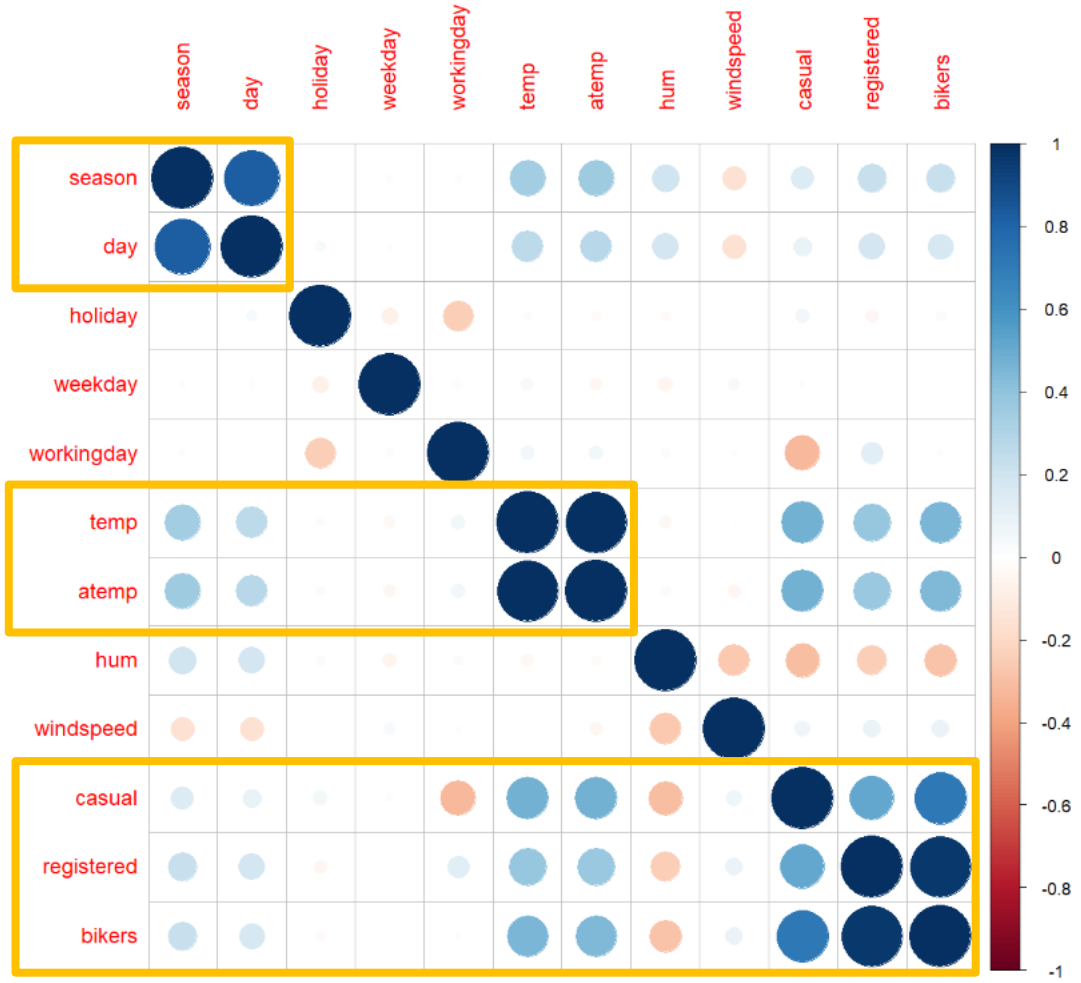


THE "BIKESHARE" DATA

Based on the definition

When faced with a large set of **<u>correlated variables</u>, principal components allow us to <u>summarize this set</u> with a smaller number of** representative **variables** that collectively explain most of the variability in the original set.

What do you expect to happen after using PCA?

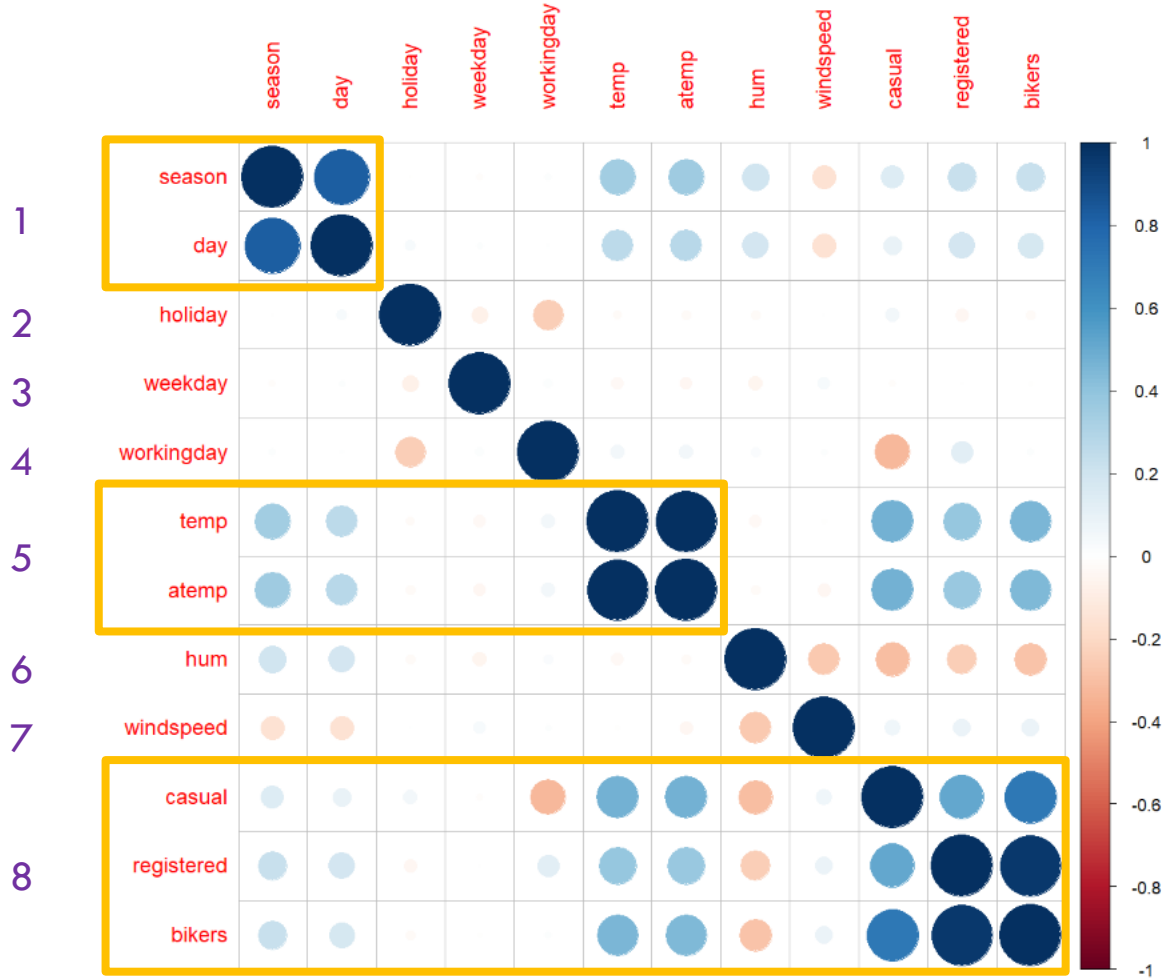


THE "BIKESHARE" DATA

Based on the definition

When faced with a large set of **<u>correlated variables</u>, principal components allow us to <u>summarize this set</u> with a smaller number of** representative **variables** that collectively explain most of the variability in the original set.

What do you expect to happen after using PCA?



LET'S APPLY PCA TO THE DATA

But first, PCA should always be performed after standardizing each variable to have mean zero and standard deviation one.

Why does it matter that we scaled the variables?

If we perform PCA on unscaled variables, then the first principal component loading vector will have a very large loading for the variable with the highest variance. This happens when the variables are measured in different units.

EXERCISE 3.1:

1. Prepare your data to be used for PCA. For this, subtract the mean and divide by the standard deviation each of the columns. Save the result in a new variable.

Hint: Use the functions:

mutate()	mean()
across()	sqrt()
everything()	var()

2. Apply PCA to the new data and save the result in a new variable.

Hint: Use the function prcom().

UNDERSTANDING THE OUTPUT

The output from prcomp() contains a number of useful variables:

- The center and scale components correspond to the means and standard deviations of the variables that were used for scaling prior to implementing PCA. This only applies if you did not standardize your variables and, therefore, you used the parameter “scale = TRUE”.
- x has as its columns the principal component score vectors. That is, the kth column is the kth principal component score vector.
- The rotation matrix provides the principal component loadings. PCA loadings (ranging from -1 to +1) represent the correlation or weight of each original variable on a principal component, showing how variables create that component. High absolute values near 1 or -1 indicate strong influence, while values near 0 mean weak influence. The sign indicates direction: same signs imply positive correlation with the component, opposite signs imply contrast.

3.2 VISUALIZING DIMENSIONALITY REDUCTION

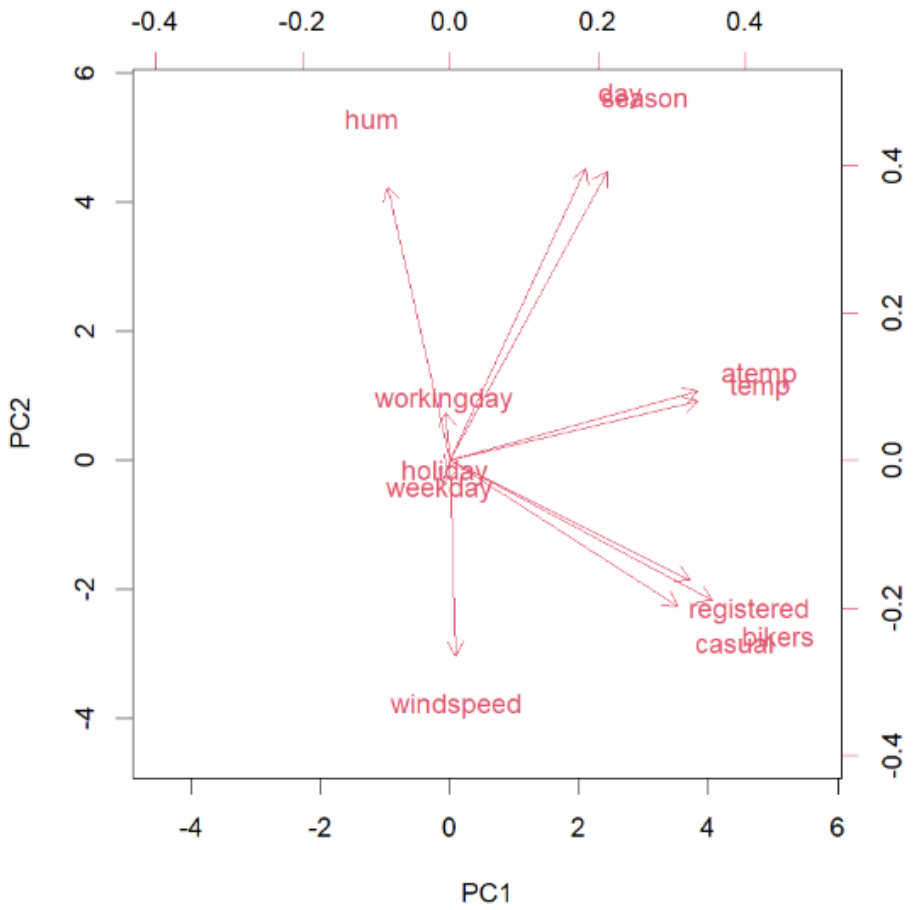
VISUALIZING PRINCIPAL COMPONENTS

To visualize the first two principal components and their loadings, you can use the function biplot().

EXERCISE 3.2:

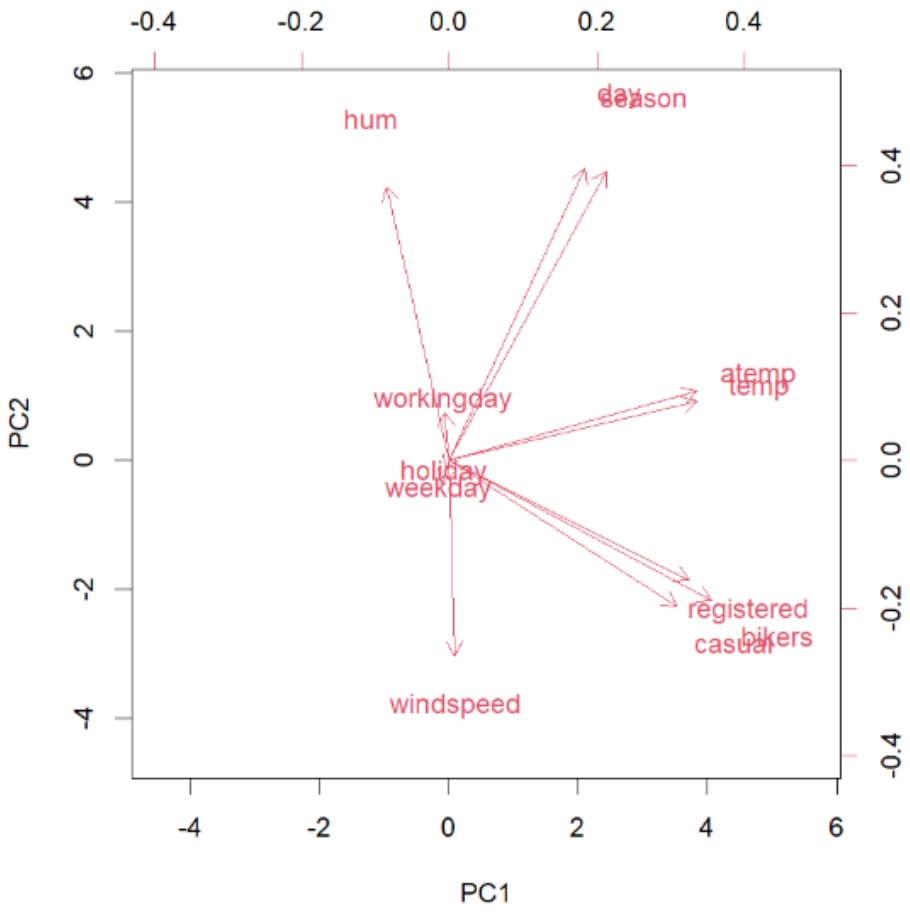
- What does the parameter scale=0 do?
- Use the parameter xlabs to remove the row ids, for this you need to create a vector of empty strings.

Hint: Use the functions rep() and nrow().

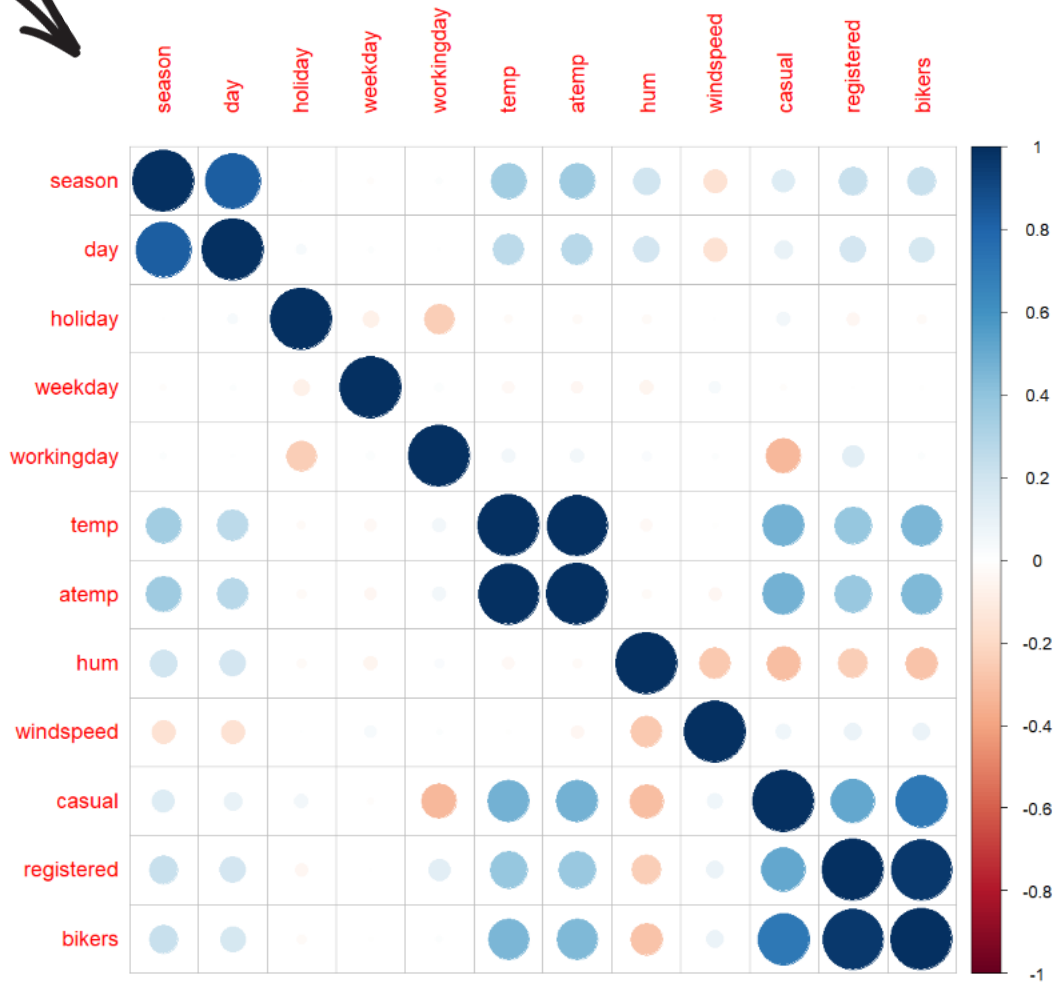


VISUALIZING PRINCIPAL COMPONENTS

Take a minute to understand the graph. How would you interpret it?



Hint



THE PROPORTION OF VARIANCE EXPLAINED (PVE)

We can now ask a natural question: how much of the information in a given data set is lost by projecting the observations onto the first few principal components? That is, how much of the variance in the data is not contained in the first few principal components? More generally, we are interested in knowing the proportion of variance explained (PVE) by each proportion of principal component.

EXERCISE 3.3:

Obtain the PVE.

1. Extract the variable sdev from the prcomp() outcome.
2. Exponentiate it element of sdev to the power of 2.
3. Divide its element by the overall sum.

LET'S WRITE OUR FIRST FUNCTION!

For this, we are going to make our first function!

```
FUNCTION NAME<- function(PARAMETERS){

                    ALGORITHM

                    return(VARIABLES) }
```

Parameters: The outcome from running the function prcomp() and the number of principal components you want to check.

Algorithm:

1. Extract the variable sdev from the prcomp() outcome.
2. Exponentiate it element of sdev to the power of 2.
3. Divide its element by the overall sum.
4. Add up the proportion of variance explained by the number of components.

Output: The cumulative proportion of variance explained by the first number of principal components.

HOW MANY PRINCIPAL COMPONENTS TO USE?

Unfortunately, there is no single (or simple!) answer to this question.

One way is by choosing the smallest number of principal components that are required in order to explain a sizable amount of the variation in the data.

This is done by checking the cumulative Proportion of the Variance Explained and looking for a point at which the proportion of variance explained by each subsequent principal component drops off.

In practice, we tend to look at the first few principal components in order to find interesting patterns in the data. If no interesting patterns are found in the first few principal components, then further principal components are unlikely to be of interest.

EXERCISE 3.4:

Based on exercise 3, how many PCs should you consider?

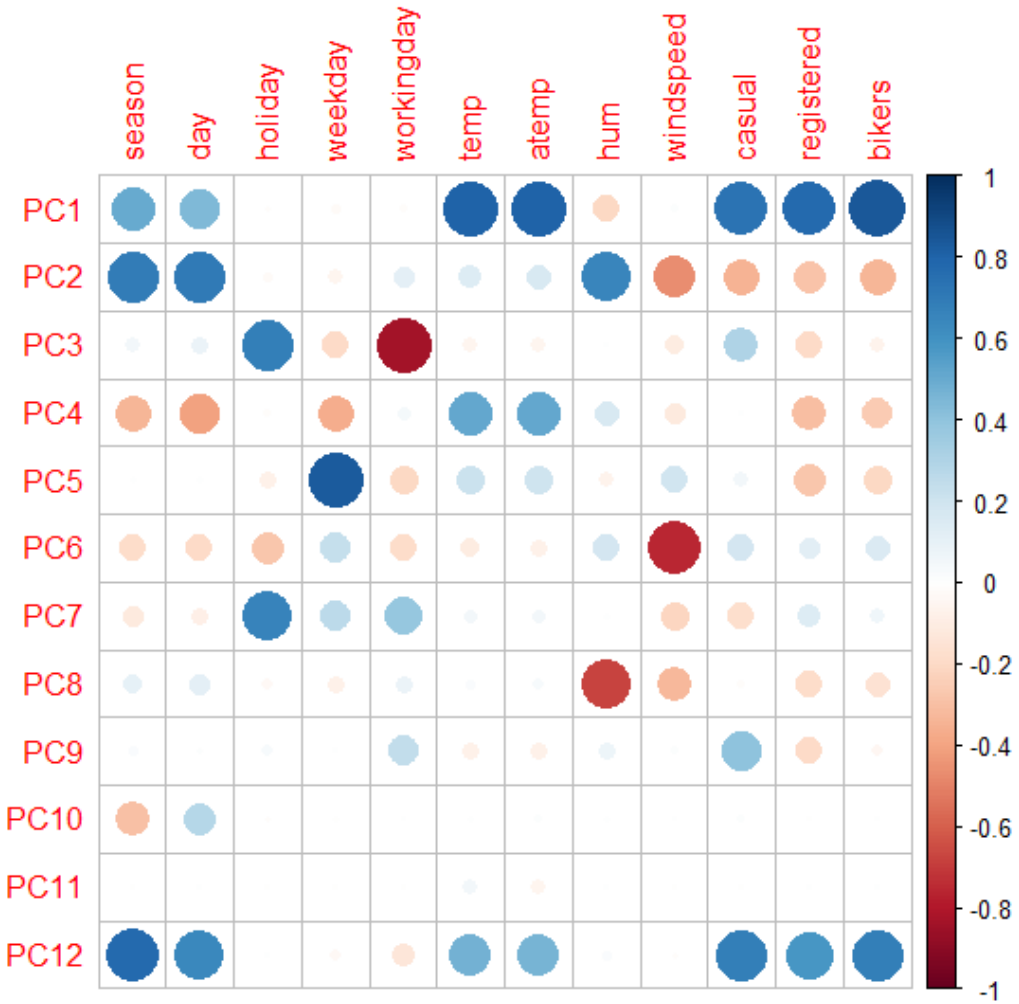
WHAT VARIABLES ARE REPRESENTED IN EACH PC?

EXERCISE 3.5:

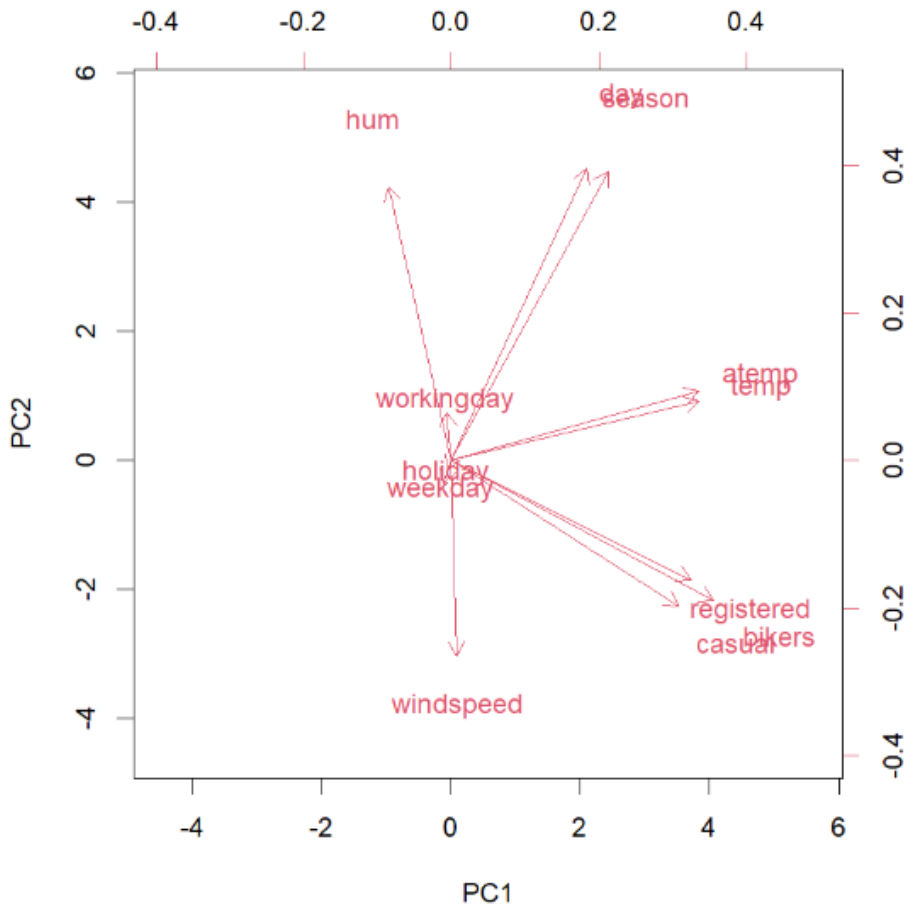
1. In a new variable, save the result of getting the correlation between the principal components and the original data.

   Hint: Use the function cor().

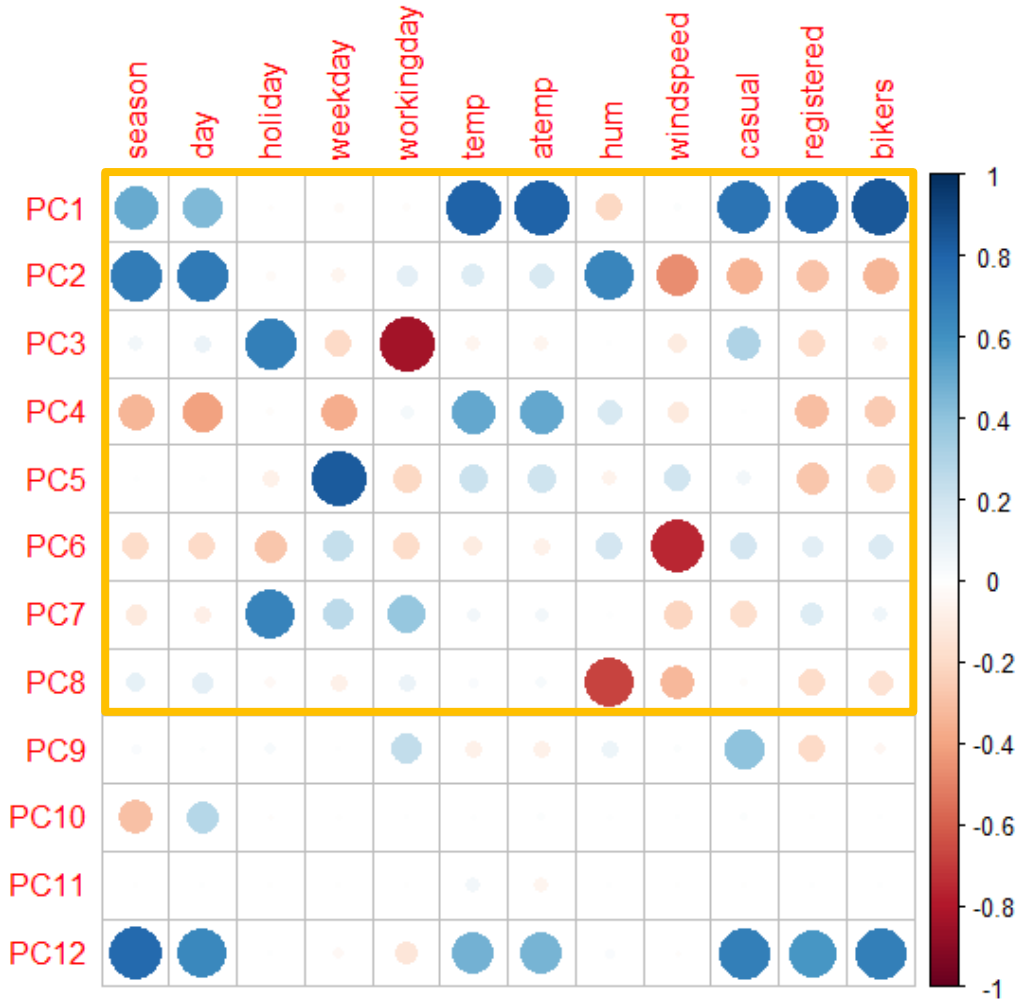
2. Visualize the results using the function corrplot().



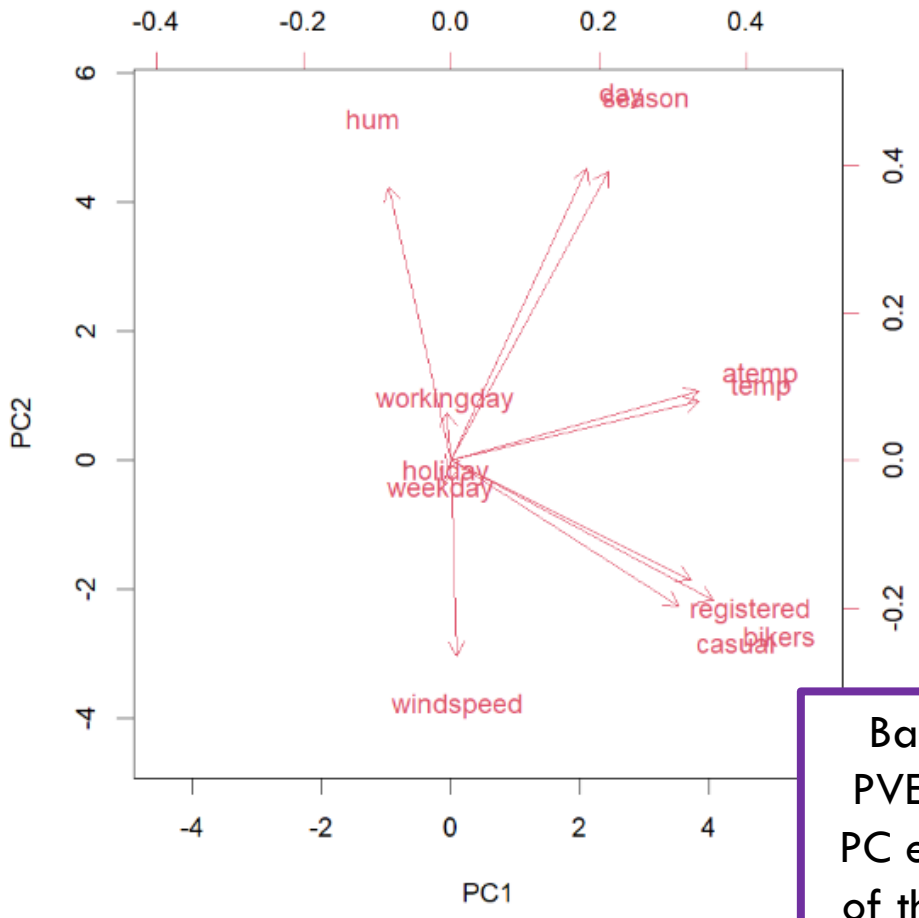
WHAT VARIABLES ARE REPRESENTED IN EACH PC?



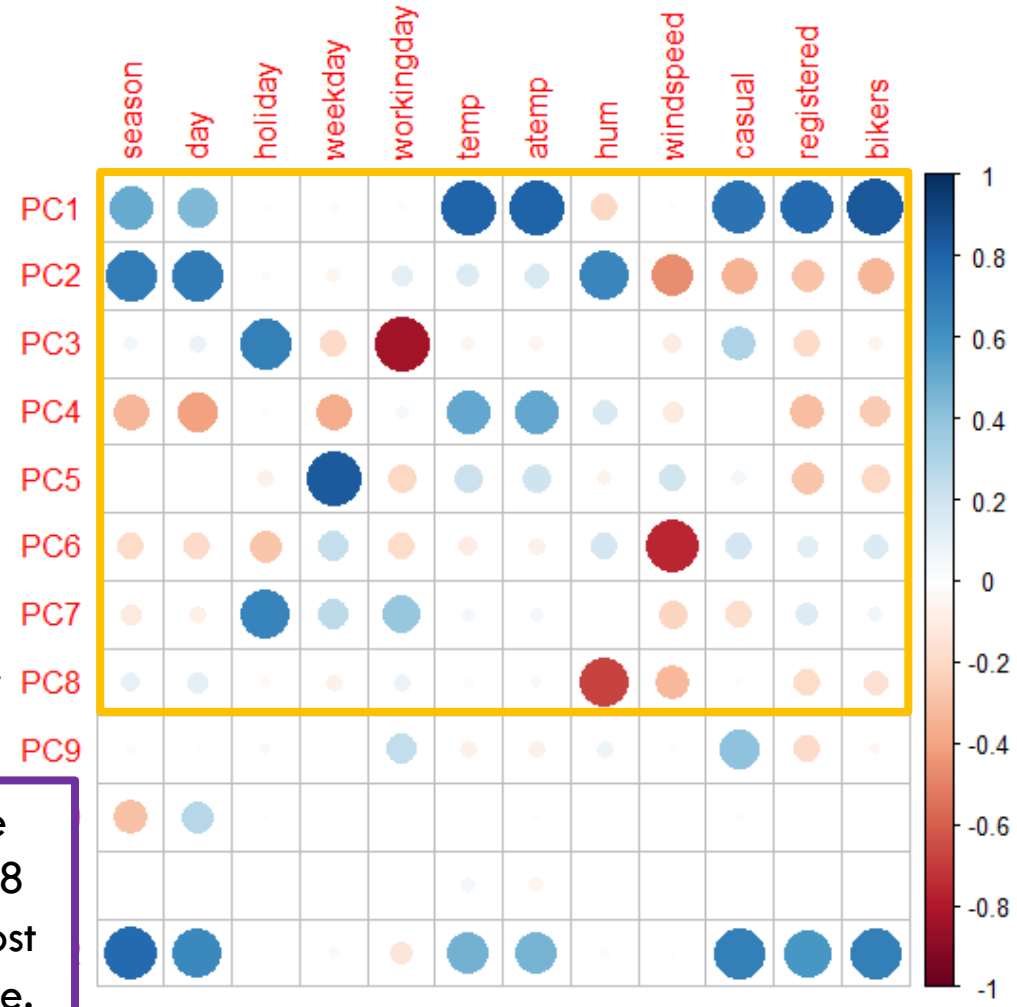
Let's check the correlations between the principal components and the original data.



WHAT VARIABLES ARE REPRESENTED IN EACH PC?

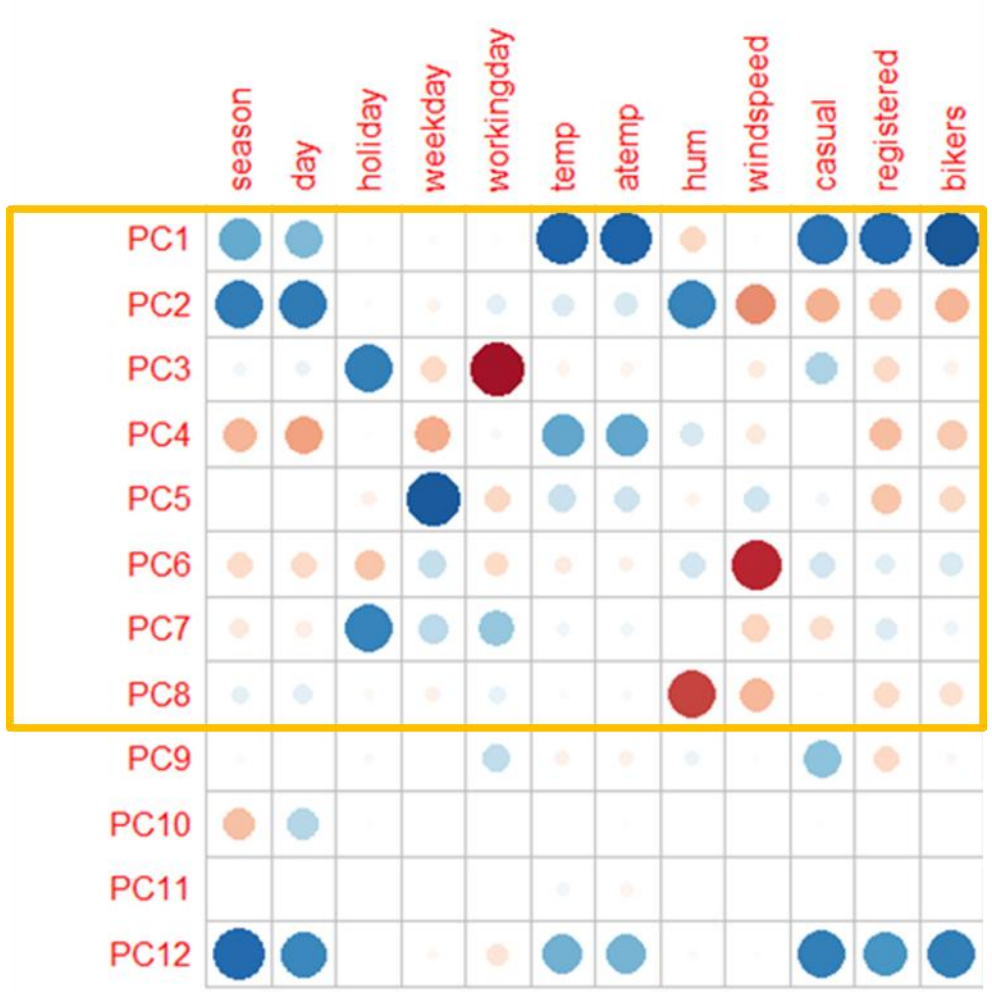


Let's check the correlations between the principal components and the original data.



Based on the PVE, the first 8 PC explain most of the variance.

WHEN COULD YOU APPLY IT?



You can analyze the most meaningful principal components instead of the original data. Though, it may be more difficult to interpret!

BREAK

See you in 10 mins.

4. K-MEANS

1. Introduction
2. Visualizing clusters
3. Making sense of the results

WHAT IS K-MEANS?

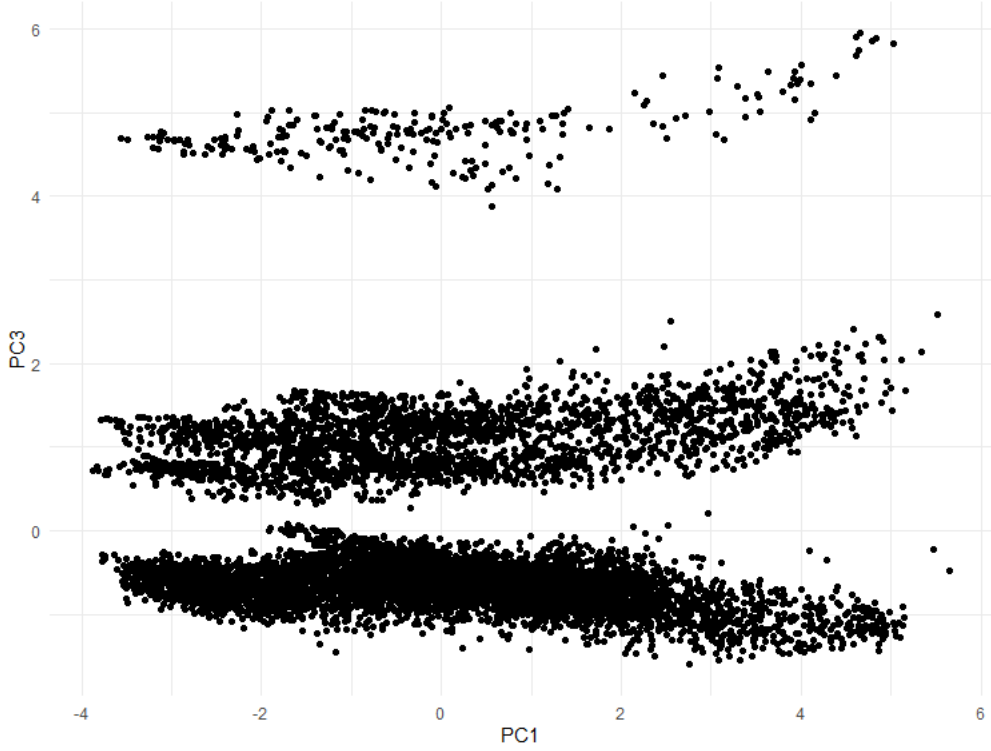
K-means clustering is a simple and elegant approach for partitioning a data set into K distinct, non-overlapping clusters. To perform K-means clustering, we must first specify the desired number of clusters K; then the K-means algorithm will assign each observation to exactly one of the K clusters.

The idea behind K-means clustering is that a good clustering is one for which the within-cluster variation is as small as possible.

LET'S USE THE FIRST 8-PC FROM BEFORE

EXERCISE 4.1:

1. Visualize different pairs of PC. Can you find something interesting?
2. Now that you have spotted a possible number of clusters, use that number to partition your data using k-means. For this, use the function kmeans().



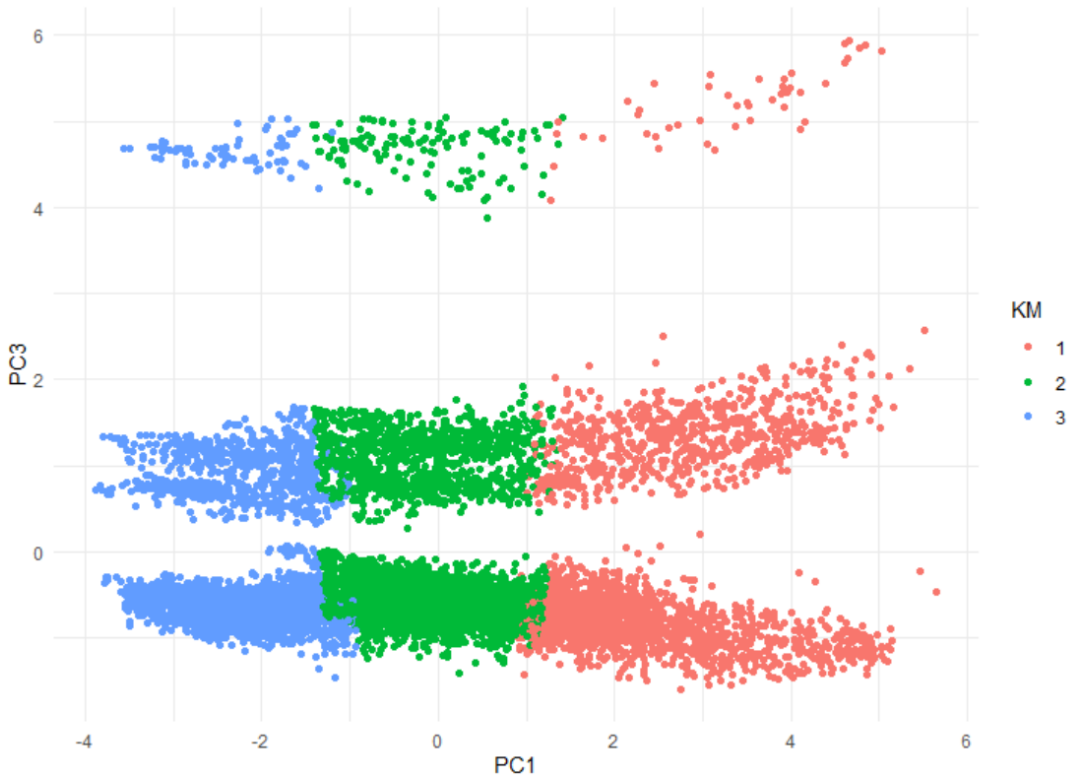
LET'S VISUALIZE THE RESULTS

EXERCISE 4.2:

1. To visualize the results, you need to add the cluster variable from the kmeans() outcome to your dataframe.
2. This variable needs to be a factor, i.e. you need to tell R that the variable is categorical.

   Hint: Use the function factor() or as.factor().
3. Now you can visualize your data using ggplot2.

Is this wrong?



WHAT IS HAPPENING? ARE THE CLUSTERS WRONG?

EXERCISE 4.3:

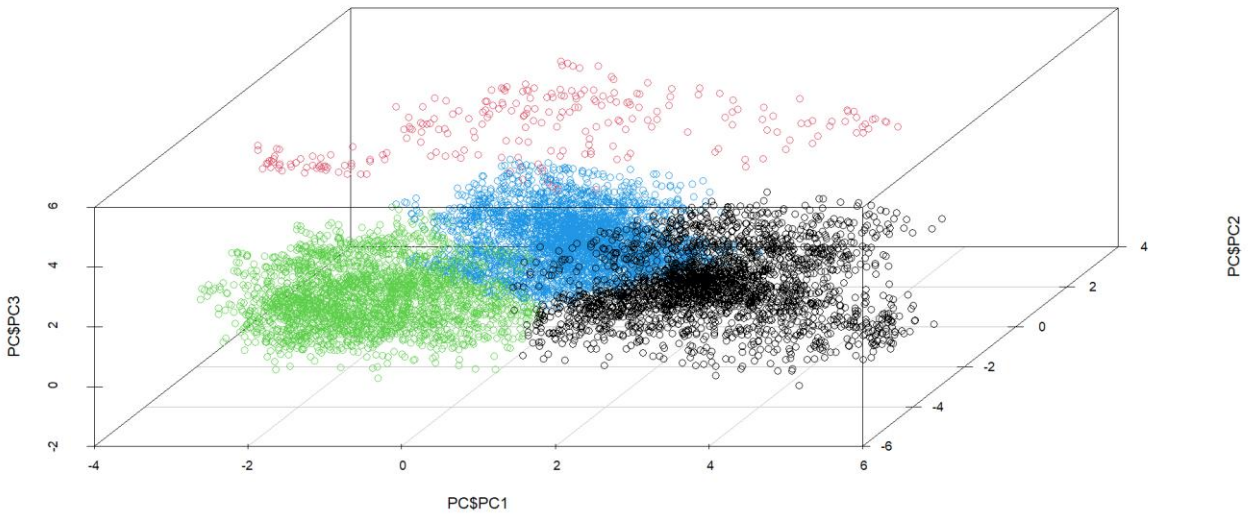
1. Let's analyze three axes instead of two. Perhaps, we will see some interesting patterns! For this, use the function scatterplot3d().

   How many clusters do you see?

2. Run the k-means function again using the new number of clusters.

3. Color the dots based on your new classification.

   Hint: In the scatterplot3d() function use the parameter color.



LET'S VISUALIZE THE RESULTS BY GROUP

EXERCISE 4.4:

1. For this, we are going to write a function "COR_DATA()".
2. Then, we will apply the function to COR_DATA()" for each of the k-means ids.

Name of the function: COR_DATA

Parameters: The original Bikeshare data and the id of the k-means cluster.

Algorithm:

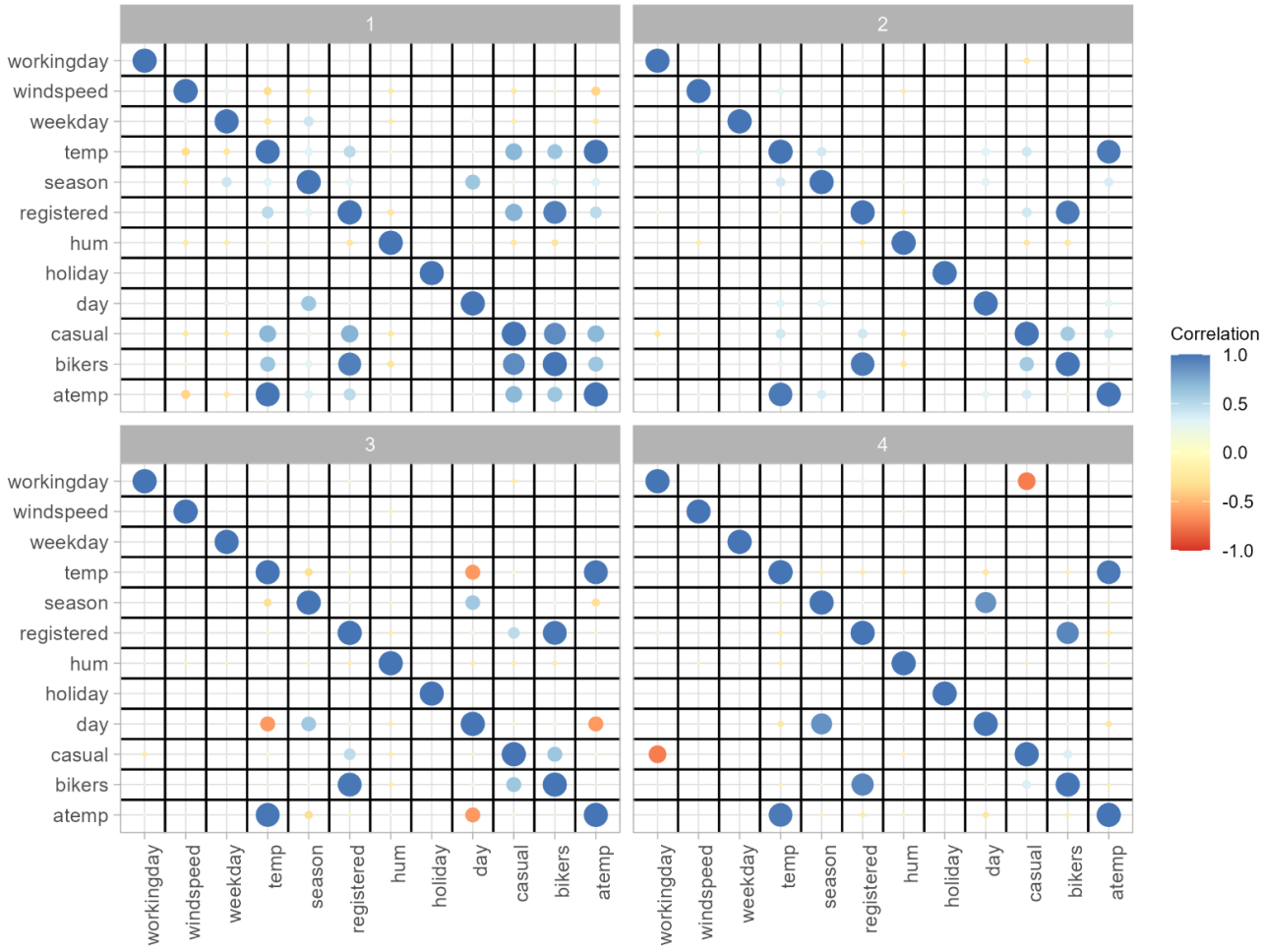
1. Keeps only the rows that correspond to the k-means id.
2. Obtains the correlation matrix.
3. To the matrix ads the variables: KM with k-mean id, and ROW with the row names. **Hint:** Use the function row.names().
4. Gathers all the columns, except the columns KM and ROW, into the variables COLUMN where the values are COR. **Hint:** Use the function gather().

Output: The new dataframe or tibble.

LET'S VISUALIZE THE RESULTS BY GROUP

EXERCISE 4.5:

1. For this, we are going to write a function "COR_DATA()".
2. Then, we will apply the function to COR_DATA()" for each of the k-means ids.
3. Then we will row bind the data frames.
4. Finally, we will create the equivalent of a circle corrplot() using ggplot and its facet_wrap() method.



Join us!

https://forms.office.com/e/8Bgd2YsasJ

All about the lab:

https://societal-analytics.nl/

Contact us at:

analytics-lab.fsw@vu.nl.

https://sofiag1l.github.io/

THANKS!

Dr. Sofia Gil-Clavel